

# Métodos de Ordenação Híbrida Adaptativa

Análise Teórica e Experimental: Timsort vs. Introsort

Italo Xavier Fonseca   João Paulo de Abreu Duarte   Matheus Tavares  
de Bessas   Thales Guerreiro

CEFET-MG - Engenharia de Computação  
Campus V - Divinópolis

Maio de 2026

- 1 Introdução Geral
- 2 O Método Timsort
- 3 O Método Introsort
- 4 Resultados Comparativos
- 5 Conclusão
- 6 Referências

# 1.1 A Evolução da Ordenação e O Objetivo do Estudo

## O Paradigma Moderno

A ordenação de dados evoluiu de algoritmos puristas para sistemas híbridos complexos. A eficiência em Big Data e a gestão de memória determinam a viabilidade de um software moderno.

- **Relevância:** O Introsort e o Timsort são os pilares da infraestrutura de TI atual. O Introsort blinda o baixo nível (C++), enquanto o Timsort organiza o alto nível (Python/Java).
- **Objetivo:** Investigar se a adaptatividade extrema do Timsort compensa o seu custo de memória face à força bruta segura e económica do Introsort, avaliando o impacto de 5 linguagens de programação distintas.

- 1 Introdução Geral
- 2 O Método Timsort
- 3 O Método Introsort
- 4 Resultados Comparativos
- 5 Conclusão
- 6 Referências

## 2.1 Introdução e Propriedades (Tim Peters, 2002)

- **A Filosofia do Mundo Real:** Dados práticos rotineiramente contêm agrupamentos internos ou subsequências já ordenadas (*natural runs*).
- **In-place (Não):** Requer a alocação de memória auxiliar de ordem  $O(n)$  para processar as mesclagens.
- **Estabilidade (Sim):** Preserva rigorosamente a organização relativa de registros com chaves idênticas.
- **Cibersegurança:** A estabilidade é crucial em perícia digital e correlação de logs de rede em ferramentas SIEM, impedindo a inversão de causa e efeito temporais.

| Melhor caso | Médio Caso    | Pior Caso     |
|-------------|---------------|---------------|
| $O(n)$ .    | $O(n \log n)$ | $O(n \log n)$ |

## 2.2 Metodologia: Runs, Minruns e Merging

O funcionamento do Timsort baseia-se na hibridização estruturada:

- ➊ **Identificação:** Mapeia sequências crescentes ou estritamente decrescentes (revertendo-as de forma linear).
- ➋ **Ajuste (*minrun*):** Estende runs artificiais curtas usando *Insertion Sort* até atingirem o limite ideal de balanceamento.
- ➌ **Pilha de Fusão:** Controla a intercalação progressiva através de regras de adjacência estritas, minimizando o esforço computacional do Merge Sort.

## 2.3 Pseudocódigo: Timsort

### Algoritmo 1: Lógica de Execução

```
Procedure Timsort(arr, n):  
    minRun = CalcularMinRun(n)  
  
    // Fase 1: Ordenar pequenas partições (runs)  
    para i = 0 até n-1 step minRun faça:  
        InsertionSort(arr, i, min(i + minRun - 1, n - 1))  
  
    // Fase 2: Mesclar runs ordenadas (Merge)  
    size = minRun  
    enquanto size < n faça:  
        para left = 0 até n-1 step 2 * size faça:  
            mid = left + size - 1  
            right = min(left + 2 * size - 1, n - 1)  
            se mid < right então:  
                Merge(arr, left, mid, right)  
        size *= 2
```

## 2.4 Desempenho Isolado: Timsort Multilinguagem

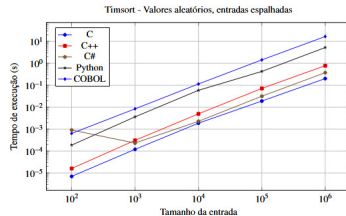


Figura 3 – Comparação do Timsort entre linguagens para valores aleatórios e entradas espalhadas

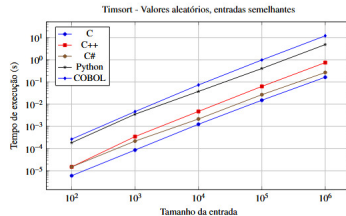


Figura 4 – Comparação do Timsort entre linguagens para valores aleatórios e entradas semelhantes



## 2.5 Análise Quantitativa: Desvios do Timsort

| Métrica (10 <sup>6</sup> Aleatório) | C        | C++             | C#       | Python   | COBOL     |
|-------------------------------------|----------|-----------------|----------|----------|-----------|
| Tempo de Execução (s)               | 0,2014 s | <b>0,7791 s</b> | 0,3742 s | 5,2421 s | 16,7020 s |
| Desvio Relativo (Base: C++)         | -74,1%   | <b>0%</b>       | -52,0%   | +572,8%  | +2.043,6% |

**Tabela:** Resultados consolidados para o Timsort em entradas espalhadas.

- **O Peso do Interpretador:** O Python sofreu um acréscimo de 572,8% em relação ao C++, impulsionado pela representação de inteiros como objetos completos em memória (PyObject).
- **Evidência Adaptativa:** O tempo geral recuou significativamente nos testes com dados semelhantes, atestando o reaproveitamento das runs.

- 1 Introdução Geral
- 2 O Método Timsort
- 3 O Método Introsort**
- 4 Resultados Comparativos
- 5 Conclusão
- 6 Referências

### 3.1 Introdução e Propriedades (David Musser, 1997)

- **A Fraqueza do Quicksort:** O Quicksort é vulnerável ao pior caso  $O(n^2)$ .
- **A Proposta:** O Introsort monitoriza a própria recursão, tornando-se imune à degradação de performance.
- **In-place (Sim):** Memória auxiliar mínima baseada na pilha de recursão ( $O(\log n)$ ).
- **Estabilidade (Não):** As partições e trocas não garantem a ordem de chaves iguais.
- **Cibersegurança:** Ao travar o pior caso quadrático, impede Ataques de Complexidade Algorítmica (DoS) voltados ao esgotamento de CPU do servidor.

| Melhor caso     | Médio Caso    | Pior Caso     |
|-----------------|---------------|---------------|
| $O(n \log n)$ . | $O(n \log n)$ | $O(n \log n)$ |

## 3.2 Metodologia: O Mecanismo de Fallback

O Introsort alterna entre três algoritmos distintos conforme as condições do vetor:

- 1 **Velocidade:** Inicia com Quicksort utilizando a estratégia de pivô por mediana de três.
- 2 **Segurança:** Se a recursão atinge o limite crítico de  $2 \times \log_2 n$ , o algoritmo muda para o **Heapsort** para assegurar o teto de  $O(n \log n)$ .
- 3 **Otimização:** Subpartições reduzidas ( $\leq 16$  elementos) finalizam via **Insertion Sort**, maximizando a eficiência de cache.

## 3.3 Pseudocódigo: Introsort

### Algoritmo 2: Procedimento com Fallback Automático

```
Procedure Introsort(arr, n):
```

```
    depthLimit = 2 * log2(n)
```

```
    IntroUtil(arr, 0, n - 1, depthLimit)
```

```
Function IntroUtil(arr, low, high, depthLimit):
```

```
    size = high - low + 1
```

```
    if size < 16: // Eficiência de Cache L1
```

```
        InsertionSort(arr, low, high)
```

```
    return
```

```
    if depthLimit == 0: // Freio de emergência Anti- $O(n^2)$ 
```

```
        Heapsort(arr, low, high)
```

```
    return
```

```
    p = Partition(arr, low, high) // Quicksort normal
```

```
    IntroUtil(arr, low, p - 1, depthLimit - 1)
```

```
    IntroUtil(arr, p + 1, high, depthLimit - 1)
```

# 3.4 Desempenho Isolado: Introsort Multilinguagem

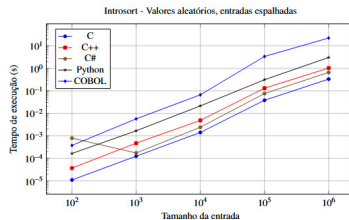


Figura 1 – Comparação do Introsort entre linguagens para valores aleatórios e entradas espalhadas

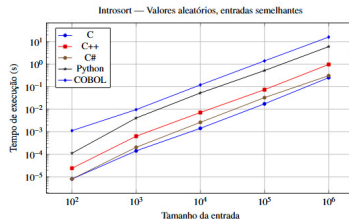


Figura 2 – Comparação do Introsort entre linguagens para valores aleatórios e entradas semelhantes

### 3.5 Análise Quantitativa: Desvios do Introsort

| Métrica (10 <sup>6</sup> Aleatório) | C        | C++             | C#       | Python   | COBOL     |
|-------------------------------------|----------|-----------------|----------|----------|-----------|
| Tempo de Execução (s)               | 0,3305 s | <b>1,0228 s</b> | 0,6571 s | 3,0210 s | 22,0600 s |
| Desvio Relativo (Base: C++)         | -67,7%   | <b>0%</b>       | -35,8%   | +195,4%  | +2.056,7% |

**Tabela:** Resultados obtidos nos apêndices para o cenário de valores espalhados.

- **Compilador JIT:** O ecossistema .NET (C#) superou o C++ nativo em 35,8% devido a otimizações dinâmicas de hardware.
- **Força Bruta:** Os benchmarks de valores semelhantes mantiveram a mesma proporção, ratificando que o Introsort não é um algoritmo adaptativo.

- 1 Introdução Geral
- 2 O Método Timsort
- 3 O Método Introsort
- 4 Resultados Comparativos**
- 5 Conclusão
- 6 Referências



## 4.1 O Custo Estrutural: Consumo de Memória

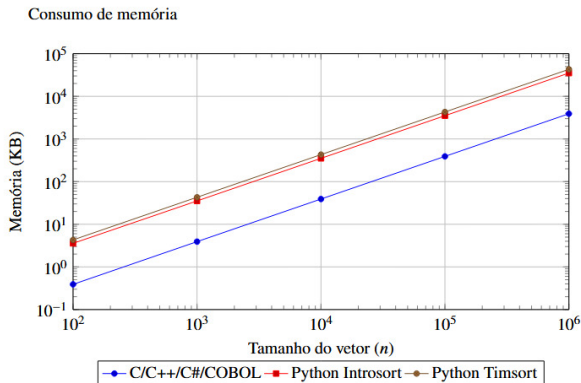
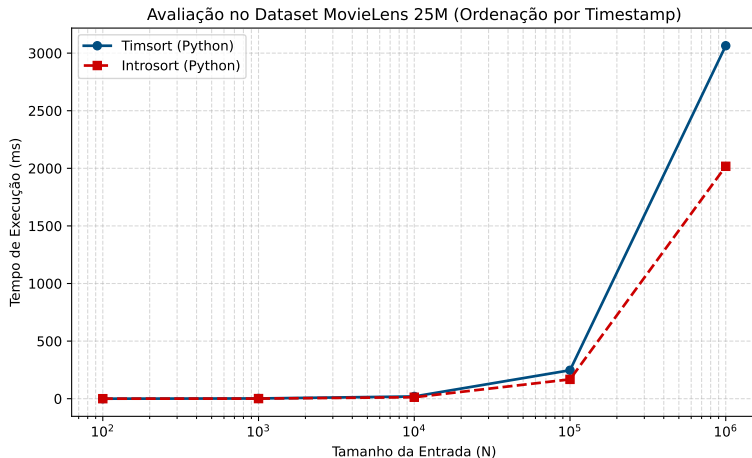


Figura 8 – Comparação do consumo aproximado de memória entre as implementações

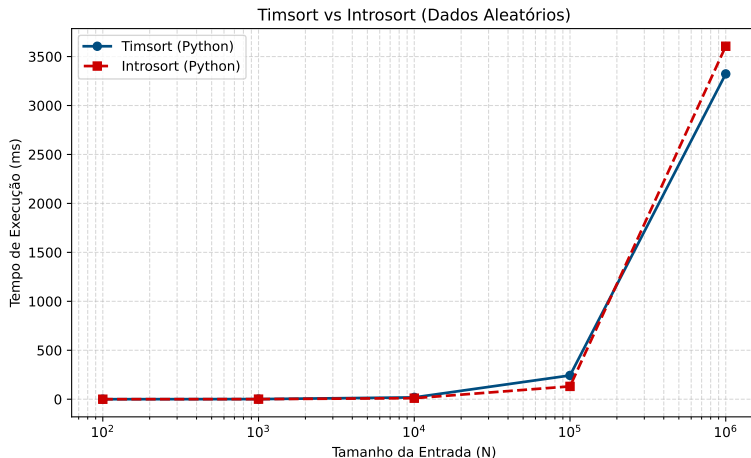
**Diagnóstico:** Alocação linear fixa em C, C++, C# e COBOL ( $\approx 3.906$  KB). O Python exigiu 35.156 KB no Introsort e escalou para 42.968 KB no Timsort devido às listas auxiliares ( $O(n)$ ).

## 4.2 Confronto Prático: Dataset MovieLens 25M



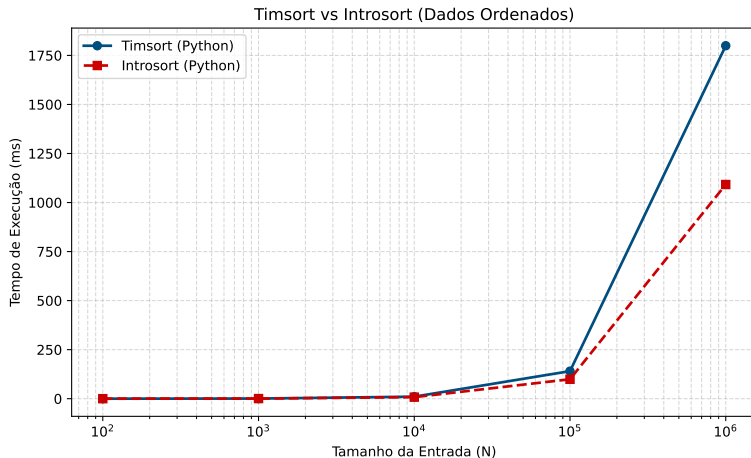
**Análise Crítica ( $N = 10^6$ ):** O Introsort finalizou em 2.021 ms contra 3.060 ms do Timsort. Sob hardware restrito (8GB RAM), o fatiamento e fusão de listas do Timsort sobrecarregaram o *Garbage Collector* do Python, enquanto o modelo *in-place* do Introsort fluiu livre desse gargalo.

## 4.3 Confronto Teórico: Cenário Aleatório



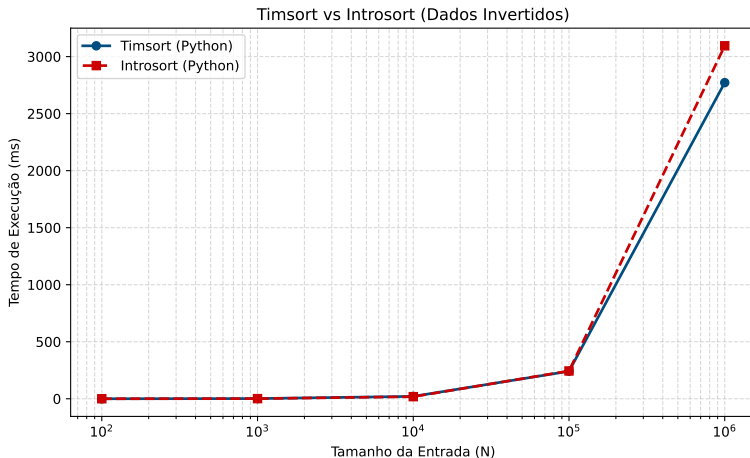
**Alta Entropia:** Sob desorganização máxima, os algoritmos operam no caso médio equilibrado. O Introsort assume ligeira vantagem (3,02 s vs 5,24 s) por não gastar ciclos de processamento tentando estruturar runs inexistentes.

## 4.4 Confronto Teórico: Cenário Ordenado



**Melhor Caso Teórico:** O Timsort mapeia o vetor em um único passe linear  $O(n)$ . Todavia, na execução puramente interpretada em Python com 1 milhão de inteiros pesados, o custo operacional do runtime gerou o comportamento observado graficamente.

## 4.5 Confronto Teórico: Cenário Invertido



**Pior Caso Estruturado:** O Timsort sobressai-se perante o Introsort (5,90 s vs 6,49 s). A engenharia do algoritmo identifica a sequência decrescente integral e reverte-a em tempo linear, poupando os sucessivos *swaps* do Quicksort.

## 4.6 Quadro Resumo Quantitativo

### Confronto Geral de Desempenho (Python, $N = 10^6$ )

*Valores consolidados diretamente das medições experimentais dos Apêndices:*

| Cenário de Teste  | Introsort | Timsort   | Impacto Relativo               |
|-------------------|-----------|-----------|--------------------------------|
| Dados Aleatórios  | 3,0210 s  | 5,2421 s  | <b>Timsort +73,5% (Lento)</b>  |
| Dados Ordenados   | 2,3717 s  | 3,8036 s  | <b>Timsort +60,3% (Lento)</b>  |
| Dados Invertidos  | 6,4942 s  | 5,9010 s  | <b>Timsort -9,1% (Rápido)</b>  |
| Dataset MovieLens | 2,0210 s  | 3,0600 s  | <b>Timsort +51,4% (Lento)</b>  |
| Consumo de RAM    | 35.156 KB | 42.968 KB | <b>Timsort +22,2% (Pesado)</b> |

**Veredicto:** O Timsort mitiga custos quando encontra estruturas ordinais prontas. Porém, sob estresse de memória, a arquitetura *in-place* do Introsort garante a estabilidade do sistema.

- 1 Introdução Geral
- 2 O Método Timsort
- 3 O Método Introsort
- 4 Resultados Comparativos
- 5 Conclusão**
- 6 Referências

## 5.1 Síntese dos Achados

- **Timsort:** Consolida-se como a melhor escolha para sistemas de alto nível (Data Science e Bancos de Dados estruturados) onde a **estabilidade** é obrigatória e a memória não é restrita.
- **Introsort:** Consolida-se como o escudo perfeito de sistemas embarcados, kernels e bibliotecas de baixo nível (como a STL do C++) devido à sua previsibilidade assintótica e economia **in-place**.
- **O Fator JIT:** A engenharia de runtimes modernos (como a CLR do C#) demonstrou capacidade de aproximar e, por vezes, superar linguagens puramente nativas em lógicas repetitivas de ordenação.



## 5.2 Limitações e Trabalhos Futuros

- **Limitações:** O uso restrito de chaves numéricas inteiras mitigou o real impacto de cópias em memória. Chaves complexas (como objetos ou strings longas) tornariam o custo de *swap* do Introsort muito mais proibitivo perante o Timsort.
- **Trabalhos Futuros:**
  - Expandir os testes para além dos limites da memória RAM avaliando algoritmos de ordenação externa e o impacto da latência de disco/SSD
  - Mapear quantitativamente as falhas de *Cache Miss* através de ferramentas de *profiling* de hardware.
  - Avaliar implementações assíncronas concorrentes em GPU via CUDA.

- 1 Introdução Geral
- 2 O Método Timsort
- 3 O Método Introsort
- 4 Resultados Comparativos
- 5 Conclusão
- 6 Referências



MUSSER, David. *Introspective Sorting and Selection*. 1997.



PETERS, Tim. *Timsort description*. 2002.



TAVARES, M. et al. *Algoritmos de ordenação Timsort e Introsort*. CEFET-MG, 2026.

# Obrigado pela atenção!

Dúvidas?